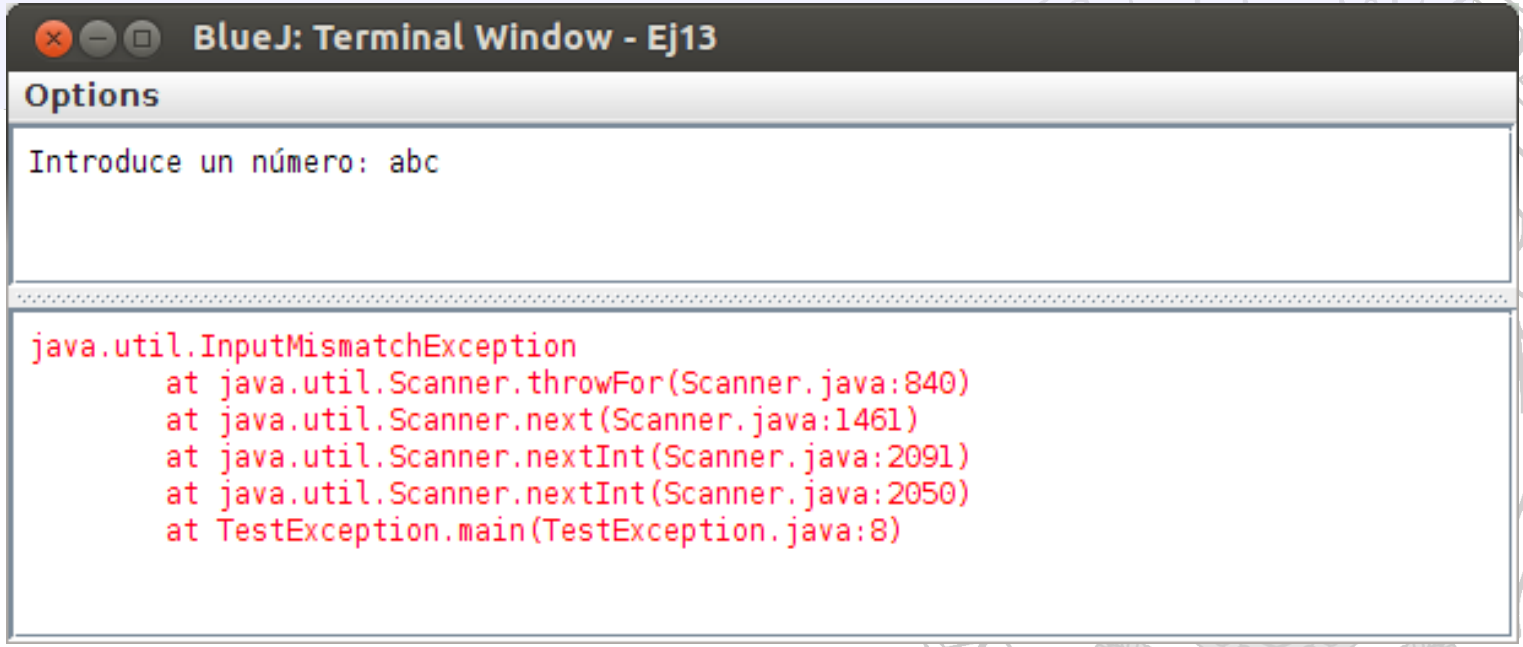
Excepciones

- Una excepción es una situación anómala que ocurre en tiempo de ejecución.
  - Intentar abrir un archivo que no existe.
  - Intentar crear un archivo sin que haya espacio suficiente en el disco.
  - Intentar leer un número del teclado, cuando el usuario ha introducido algo que no es un número.
  - Intento de acceder a un array más allá de sus límites.
  - etc.
- Java permite al programador tomar el control del programa en presencia de estas situaciones anómalas → capturar una excepción.

Generación de excepciones

```
public class TestException
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce un número: ");
        int a = sc.nextInt();
        System.out.println("El número que has introducido es " + a);
    }
}
```

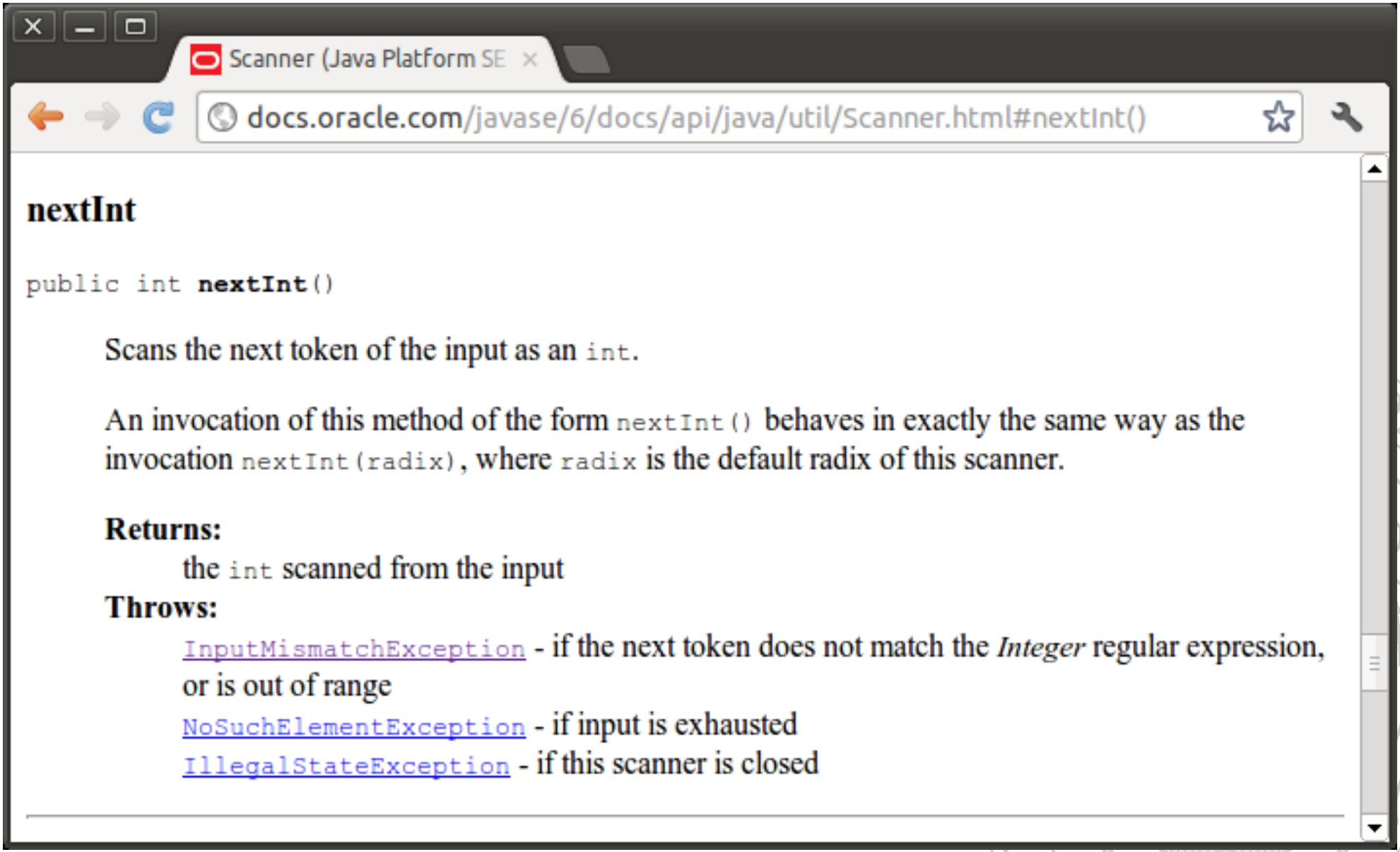


Clases de excepciones

- El mensaje de error generado por la JVM no sólo muestra la secuencia de llamadas en las que se produce la excepción.
- También muestra el tipo de excepción que se ha generado.

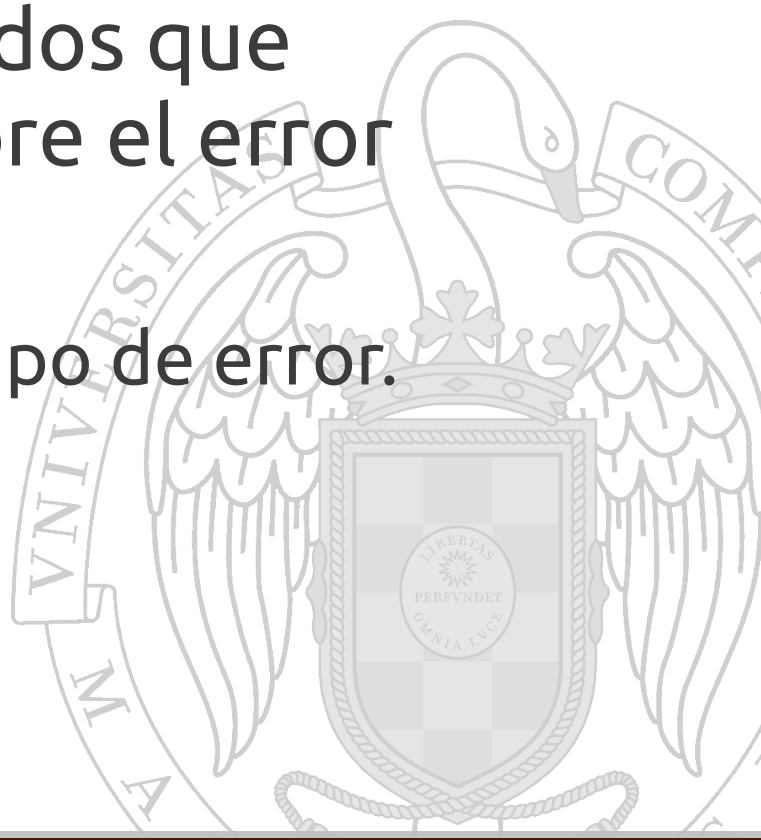
```
java.util.InputMismatchException
        at java.util.Scanner.throwFor(Scanner.java:840)
        at java.util.Scanner.next(Scanner.java:1461)
        at java.util.Scanner.nextInt(Scanner.java:2091)
        at java.util.Scanner.nextInt(Scanner.java:2050)
        at TestException.main(TestException.java:8)
```

Clases de excepciones



Clases de excepciones

- Las excepciones en Java son objetos.
- La información mostrada por la JVM es la clase a la que pertenece la excepción lanzada.
- Las excepciones poseen métodos que proporcionan información sobre el error producido.
  - Esta información depende del tipo de error.



Captura de excepciones

- Una excepción puede capturarse mediante la construcción try-catch.

```
public class TestException {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
           System.out.print("Introduce un número: ");
           int a = sc.nextInt();
           System.out.println("El número que has introducido es " + a);
        } catch (InputMismatchException e) {
           System.out.println("La cadena introducida no es un número.");
           System.out.println(e.getMessage());
        }
    }
}
```

Captura de excepciones

- Dentro del bloque `try` se coloca el código que es susceptible de generar una excepción.

```
public class TestException {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
           System.out.print("Introduce un número: ");
           int a = sc.nextInt();
           System.out.println("El número que has introducido es " + a);
        } catch (InputMismatchException e) {
           System.out.println("La cadena introducida no es un número.");
           System.out.println(e.getMessage());
        }
    }
}
```

Captura de excepciones

- Si el código dentro del bloque `try` produce una excepción, se abandona dicho bloque y se comprueba si la clase de la misma se corresponde con (o es subclase de) la clase indicada tras el `catch`.

```
public class TestException {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
           System.out.print("Introduce un número: ");
           int a = sc.nextInt();
           System.out.println("El número que has introducido es " + a);
        } catch (InputMismatchException e) {
           System.out.println("La cadena introducida no es un número.");
           System.out.println(e.getMessage());
        }
    }
}
```

Captura de excepciones

- Si la excepción es compatible con el tipo indicado tras el `catch`, se ejecutará el bloque correspondiente.
- Además, la variable indicada tras el tipo contendrá el objeto que representa la excepción lanzada.

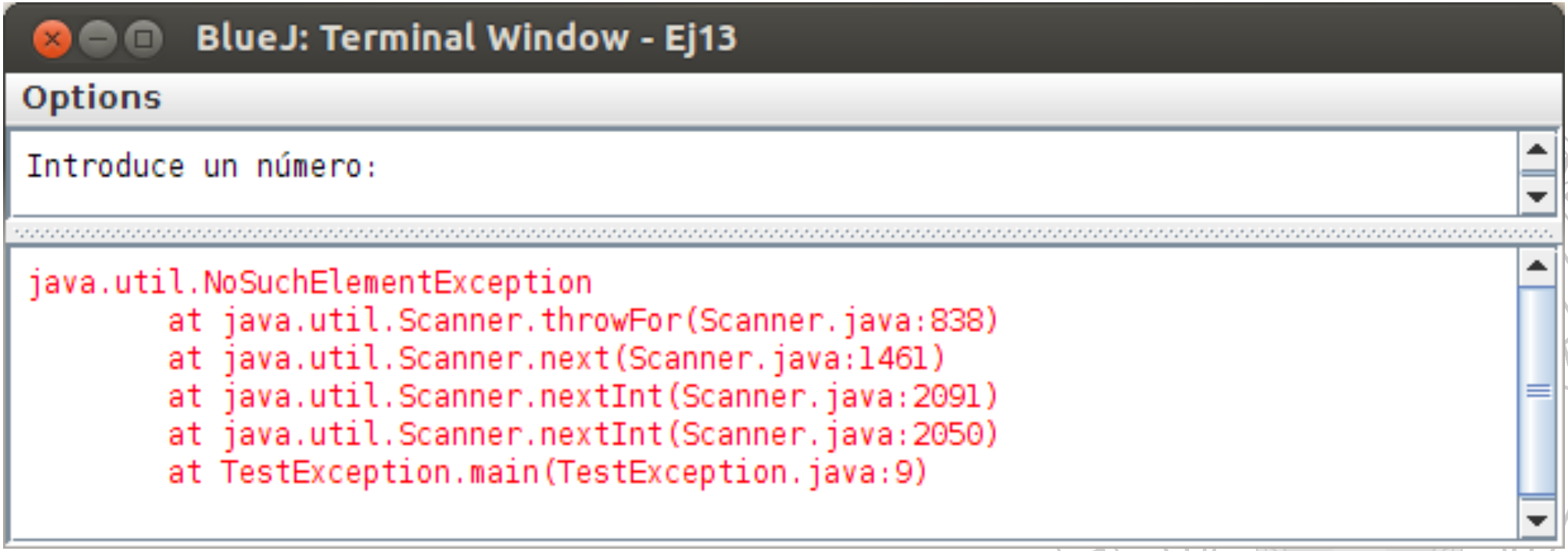
```
public class TestException {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
           System.out.print("Introduce un número: ");
           int a = sc.nextInt();
           System.out.println("El número que has introducido es " + a);
        } catch (InputMismatchException e) {
           System.out.println("La cadena introducida no es un número.");
           System.out.println(e.getMessage());
        }
    }
}
```

Captura de excepciones

- Si la excepción no es compatible con el tipo indicado tras el `catch`, se volverá a lanzar al nivel superior (la JVM).



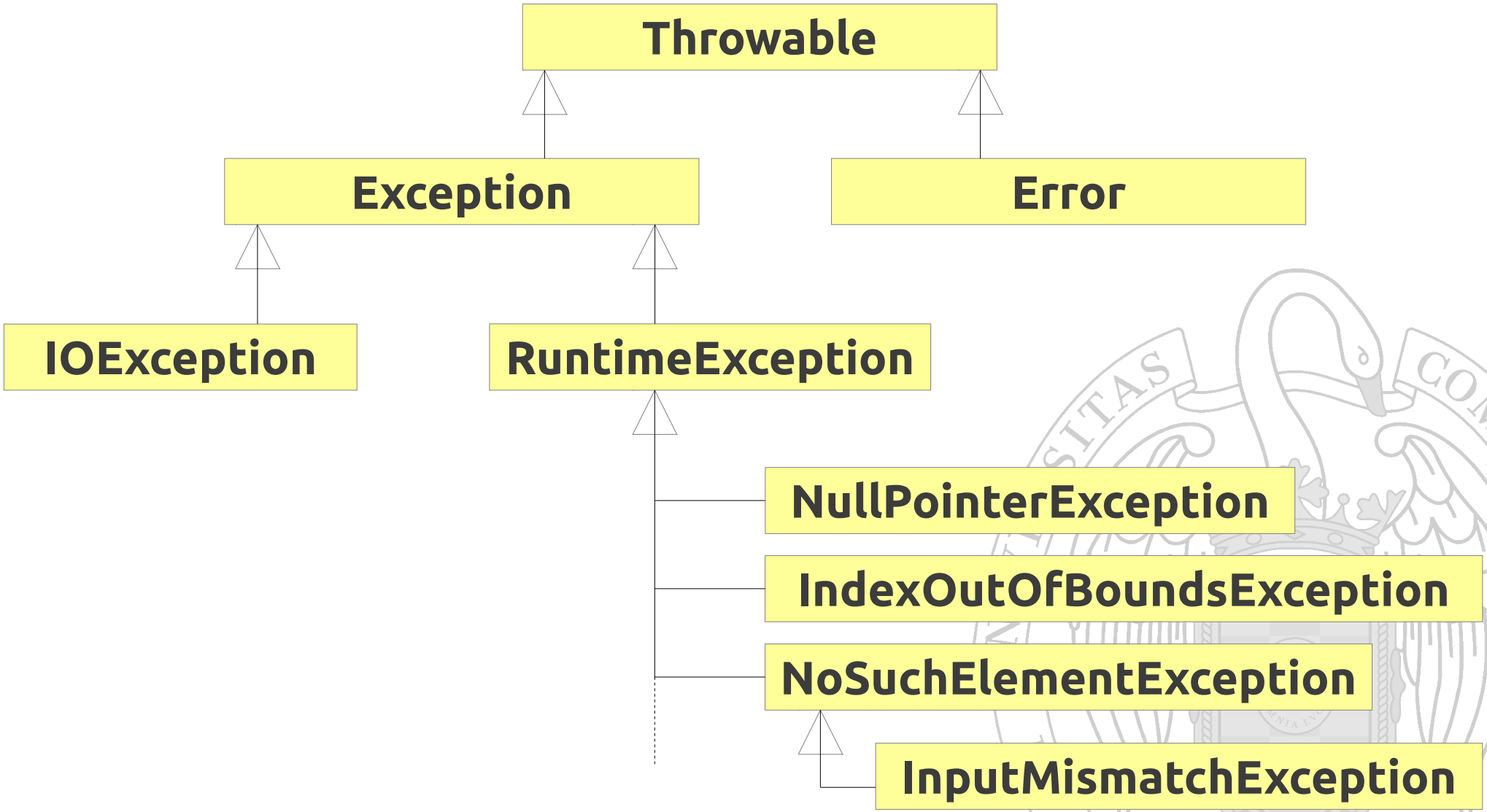
Captura de excepciones

- Es posible tener varias sentencias `catch` en un mismo bloque `try`, con el fin de capturar distintos tipos de excepciones.

```
Scanner sc = new Scanner(System.in);

try {
   System.out.print("Introduce un número: ");
   int a = sc.nextInt();
   System.out.println("El número que has introducido es " + a);
} catch (InputMismatchException e) {
   System.out.println("La cadena introducida no es un número.");
   System.out.println(e.getMessage());
} catch (NoSuchElementException e) {
   System.out.println("No hay más caracteres en la entrada.");
   System.out.println(e.getMessage());
}
```

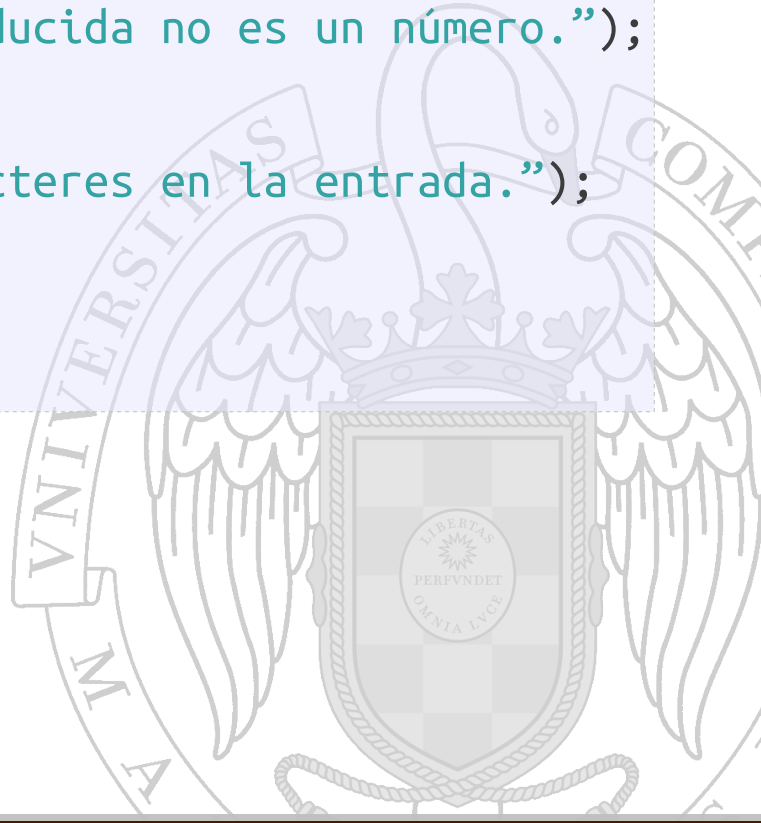
Jerarquía de excepciones



Jerarquía de excepciones

```
Scanner sc = new Scanner(System.in);

try {
   System.out.print("Introduce un número: ");
   int a = sc.nextInt();
   System.out.println("El número que has introducido es " + a);
} catch (InputMismatchException e) {
   System.out.println("La cadena introducida no es un número.");
   System.out.println(e.getMessage());
} catch (NoSuchElementException e) {
   System.out.println("No hay más caracteres en la entrada.");
   System.out.println(e.getMessage());
} catch (Exception e) {
   System.out.println(e.getMessage());
}
```



Generar una excepción

```
public class Estudiante extends Persona {
   private int puntuacionTotal;
   private int numeroCalificaciones;

   …
   public double getNotaMedia() {
      return ((double) puntuacionTotal)/numeroCalificaciones;
   }
}
```

¿Y si numeroCalificaciones = 0?

- Para generar una excepción, hay que:
  - Crear el objeto que contenga la excepción (de alguna subclase de `Exception`).
  - Lanzar dicho objeto al llamante mediante `throw`.

Generar una excepción

```
public class Estudiante extends Persona {

  public double getNotaMedia() throws Exception {
    if (numeroCalificaciones == 0)
      throw new Exception("No hay calificaciones");
    return ((double) puntuacionTotal)/numeroCalificaciones;
  }
}
```

```
Estudiante est = new Estudiante("David Sánchez", new Fecha(10, 3, 2002), 23155512);
int notaMedia = 0;
try {
  notaMedia = est.getNotaMedia();
} catch (Exception e) {
  System.out.println(e.getMessage());
}
```

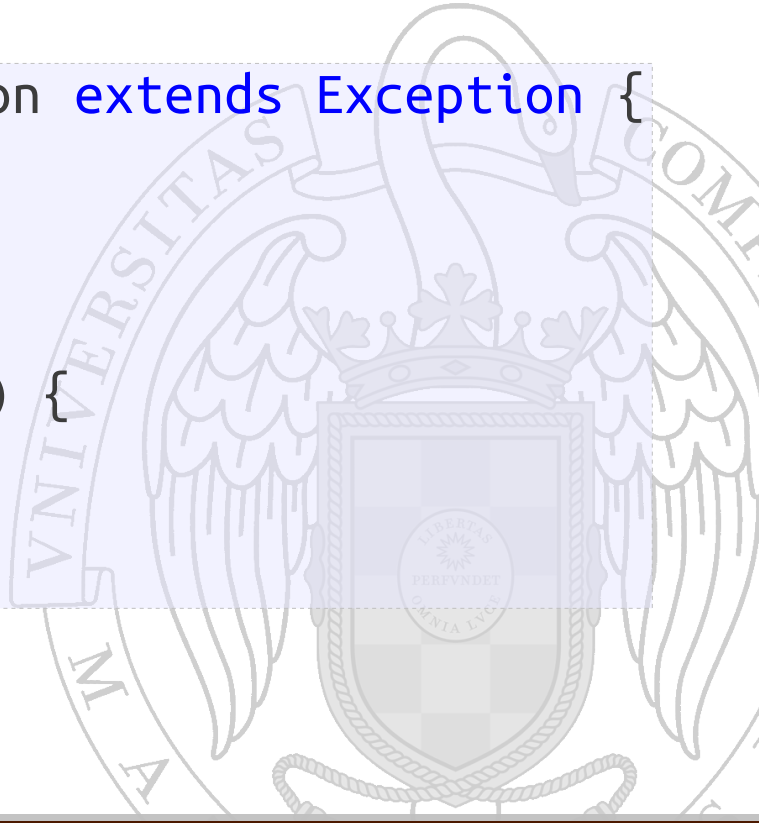


Definir un tipo de excepción

- Si no hay ninguna subclase de `Exception` que se adapte a la excepción que queremos lanzar, podemos definir nuestra propia subclase.

```
public class NoHayCalificacionesException extends Exception {
  public NoHayCalificaciones() {
    super("No hay calificaciones");
  }

  public NoHayCalificaciones(String msg) {
    super(msg);
  }
}
```



Definir un tipo de excepción

```
public class Estudiante extends Persona {

  public double getNotaMedia() throws NoHayCalificacionesException {
    if (numeroCalificaciones == 0)
      throw new NoHayCalificacionesException();
    return ((double) puntuacionTotal)/numeroCalificaciones;
  }
}
```

```
Estudiante est = new Estudiante("David Sánchez", new Fecha(10, 3, 2002), 23155512);
int notaMedia = 0;
try {
  notaMedia = est.getNotaMedia();
} catch (NoHayCalificacionesException e) {
  System.out.println(e.getMessage());
}
```



Bloques try/catch/finally

- Las operaciones en el bloque `finally` se realizarán siempre al abandonar el bloque try/catch, independientemente de si se lanza una excepción o no.
- Se utiliza para liberar recursos (cerrar archivos abiertos, por ejemplo)

```
try {
  …
} catch (IOException e) {
  …
} finally {
  // Esto siempre se ejecutará
}
```

